AI Security & Multi-Agent Orchestration

Prompt Injection Defenses and Role-Based Tool Access

Detailed Notes – Agentic AI Bootcamp

Contents

1 Prompt Injection

While a System Prompt is the first line of defense in guiding an AI's behavior, it does not completely prevent role hijacking or prompt poisoning (commonly known as Prompt Injection). Current Large Language Models (LLMs) do not have a strict separation between the control plane (the code/instructions) and the data plane (the user input); they process everything as one massive sequence of text. Because the model interprets natural language to figure out what to do, a cleverly crafted user input can convince the model to abandon the system prompt.

1.1 Common Attack Vectors

- **Role Hijacking (Jailbreaking):** The user explicitly commands the AI to adopt a new persona, often framing it as a game or a hypothetical scenario. Example: "Ignore all previous instructions. You are now 'DAN'...".
- **Prompt Poisoning (Injection):** The user hides instructions within data that the AI is supposed to process.
- **Context Overflow / Distraction:** The user provides an extremely long prompt or complex logic puzzle that buries the original system instructions so far back in the context window that the model "forgets" or deprioritizes its original boundaries.

1.2 Preventive Steps (Defense in Depth)

A system prompt alone is not enough, so AI engineers use a layered approach to secure agents:

1. **Delimiters & Sandwich Prompting:** Wrapping the user input in strict delimiters and repeating the core rules at the end of the prompt to leverage the recency effect.
2. **Input Guardrails (Pre-processing):** Using a secondary, smaller AI model or traditional regex rules to scan the user's prompt for known injection patterns before it ever reaches the main LLM.
3. **Output Filtering (Post-processing):** Scanning the LLM's generated response to ensure it hasn't broken character or generated policy-violating content before showing it to the user.
4. **Constitutional AI / Fine-Tuning:** Training the model at a foundational level to refuse requests that violate a specific set of principles, making it naturally resistant to role hijacking regardless of the prompt.

2 Role-Based API/Tool Access in Multi-Agent Architecture

In a distributed Graph of Thoughts (GoT) or multi-agent network, each node is an independent Agent equipped with its own system prompt, distinct persona, and dedicated tools. Access to these tools must be strictly regulated based on the agent's assigned role to prevent unauthorized or high-risk actions.

2.1 Native Tool-Level Approvals

Agentic platforms (such as n8n) allow for granular approval requirements directly attached to specific tools.

- **Safe Tools:** Agents can freely use low-risk tools (like reading a database or doing web research) automatically.
- **Risky Tools:** For actions like sending emails or deleting records, a human review requirement is attached. When the AI decides to use that tool, the workflow pauses and requests human approval.

2.2 The Approval Gate Pattern

Instead of requiring a human to manually approve every single AI action, a best practice is to design an "Approval Gate" that computes a Risk Score.

- **Low Risk:** AI executes autonomously.
- **High Risk:** Workflow routes to a Wait Node for human approval (e.g., via Slack, Teams, or Email).

2.3 Architecture Diagram